基于 CPP 实现的 Spider 1.0 Text2SQL 自然语言 SQL 查询 Agent

> **建议阅读 / 汇报时长：3～5 分钟**
> GitHub：https://github.com/Lx6uo/agentic-text2sql-cpp

摘要： Spider 1.0 要求模型面对不同领域的数据库，将自然语言问题转换为 SQL。围绕这一任务，本文使用 C++20 实现了一个 Agentic Text-to-SQL 命令行工具。Agent 可动态读取 SQLite Schema、生成并安全执行 SQL，并利用数据库错误进行有限修正。项目提供离线 Demo、真实 DeepSeek Tool Calling、只读安全控制和 Spider Baseline/Agentic 对照评测。20 个数据库的阶段性实验中，Agentic SQL 执行成功率为 100%，本地结果一致率为 65%。

关键词： Spider 1.0；Text-to-SQL；C++20；SQLite；Agent；Tool Calling

1. 研究背景与 Spider 1.0 数据集

1.1 Text-to-SQL 问题

数据库通常依赖 SQL 查询，但普通用户需要了解语法、表名、字段和关联关系。大模型可以从自然语言生成 SQL，但在陌生数据库中容易出现：

- 字段或表名幻觉；
- 关联条件错误；
- 输出列、行数与问题不一致；
- 生成写入或其他危险操作。

Text-to-SQL 的目标是：

```
自然语言问题 + 数据库结构 → 正确、可执行的 SQL
```

1.2 Spider 1.0

Spider 1.0 是跨领域 Text-to-SQL 数据集，由自然语言问题、数据库和标准 SQL 组成。

数据集内容	作用
`question`	用户自然语言问题
`db_id`	问题对应的数据库
SQLite 数据库	SQL 的真实执行环境
`query`	Gold SQL，用于结果评价
`tables.json`	表、字段、主键、外键

Spider 1.0 官方规模：**10,181 个问题、5,693 条 SQL、200 个数据库、138 个领域**。本项目重点使用 dev 集：**1,034 个问题、20 个数据库**。

1.3 数据集如何驱动本项目

Spider 的任务结构直接决定了项目能力：

```
多数据库、不同 Schema
        ↓
程序不能写死字段，必须动态探索

自然语言 + Gold SQL + SQLite
        ↓
既要生成 SQL，也要真实执行并比较结果

复杂 SQL 与跨表关系
        ↓
需要 JOIN、聚合、子查询和错误修正能力
```

因此，本项目不是面向某一个 Demo 数据库，而是基于 Spider 跨数据库任务实现一个可实际使用的查询 Agent。

2. 项目内容与目标

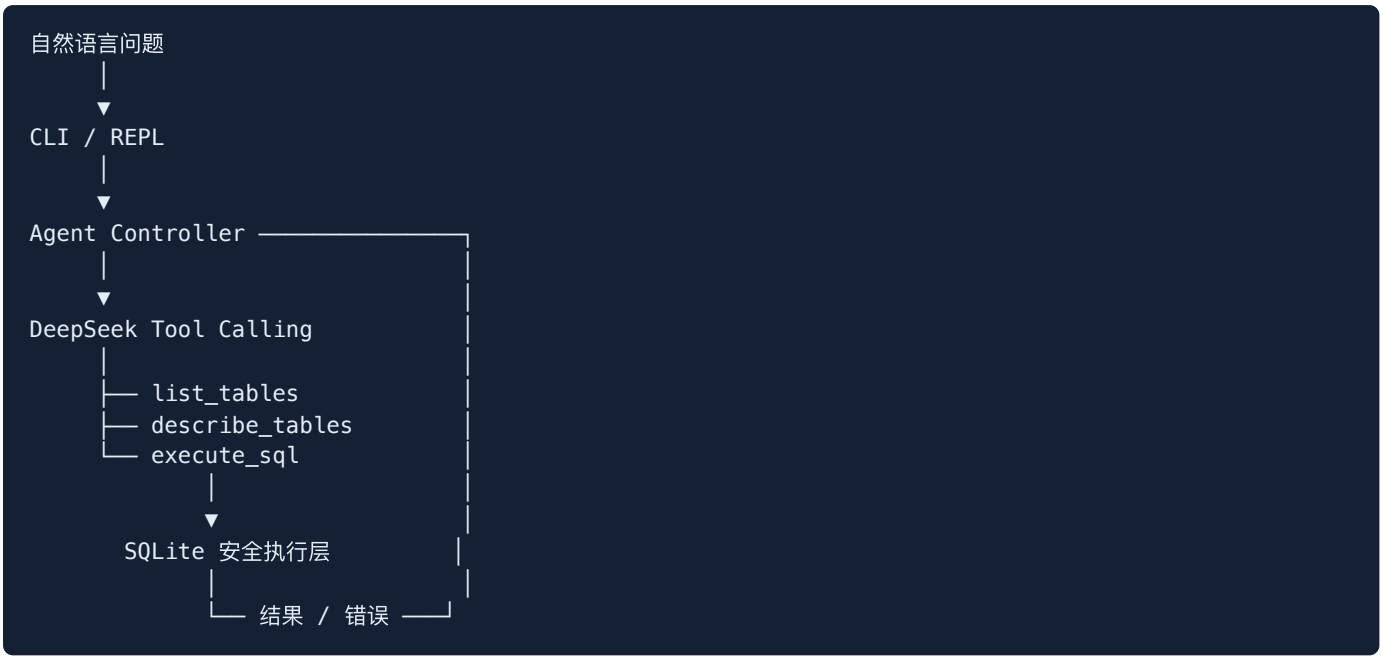
项目目标：开发一个轻量、免训练、跨数据库的 C++ 自然语言 SQL 查询工具。

目标	实现方式
🧠 理解自然语言	DeepSeek `deepseek-flash`
🔍 适配不同 Schema	Agent 动态调用数据库工具
🗄️ 执行真实 SQL	SQLite 只读连接
🔄 处理错误	将 SQLite 错误反馈给模型
🔒 保证安全	C++ / SQLite 执行层强制限制
📊 数据集评测	Spider Baseline / Agentic 对照
🖥️ 跨平台	Windows、macOS、Linux

项目不训练模型、不部署本地模型、不依赖数据库服务器，也不引入 LangChain、Web 服务或多 Agent 框架。

3. Agent 设计与实现

3.1 系统架构



3.2 三个核心工具

工具	功能
`list_tables`	获取真实表和视图
`describe_tables`	获取字段、类型、主键和外键
`execute_sql`	安全执行一条只读 SQL，返回结果或错误

Agent 自主决定查看哪些表、何时执行 SQL、是否根据错误修正。默认最多 **8 轮模型调用、3 次 SQL、2 次修正**，避免无限循环。

3.3 SQLite 安全层

模型只负责提出 SQL，最终权限由程序控制：

- `SQLITE_OPEN_READONLY`：文件只读打开；
- `sqlite3_stmt_readonly`：拒绝非只读语句；
- `sqlite3_set_authorizer`：拒绝写入、DDL、ATTACH、危险 PRAGMA；
- 拒绝多条 SQL，禁用扩展加载；
- 限制 SQL 长度、查询时间、结果行数和结果大小；
- Ctrl+C 可取消。

> **LLM 生成 SQL，不等于 LLM 获得数据库权限。**

3.4 工程实现

- C++20 + CMake；
- SQLite 3.50.4；

- nlohmann/json 3.12.0；
- libcurl HTTPS 网络层；
- REPL、单次查询、verbose、JSON 输出；
- 原生支持 Windows / macOS / Linux。

4. 运行效果

4.1 终端自然语言查询

```
$ ./build/text2sql --db data/company.sqlite --verbose
Agentic Text-to-SQL
Model: deepseek-flash
Type /help for commands.

> 查询平均工资最高的三个部门，返回部门名称、平均工资和员工人数
[1] list_tables: Success
[2] describe_tables: Success
[3] execute_sql: Success

SQL:
SELECT d.name AS 部门名称, AVG(e.salary) AS 平均工资,
       COUNT(e.id) AS 员工人数
FROM employees e
JOIN departments d ON e.department_id = d.id
GROUP BY d.id, d.name
ORDER BY 平均工资 DESC
LIMIT 3

Results:
部门名称      平均工资   员工人数
Engineering  115000.0  2
Finance      95000.0   2
Marketing    82500.0   2

Agent steps: 4, SQL attempts: 1, repairs: 0
>
```

4.2 离线错误修正

```
[1] list_tables: Success
[2] describe_tables: Success
[3] execute_sql: Failed — no such column: department_name
[4] execute_sql: Success

Agent steps: 5, SQL attempts: 2, repairs: 1
```

即使没有 API Key，Mock 模型也会触发真实 SQLite Schema 读取、SQL 失败和修正后执行。

5. Spider 实验与效果

5.1 实验设置

从 Spider dev 的 **20 个数据库各取 1 题**，覆盖 20 种不同 Schema，共执行：

- 20 个 Baseline 任务；

- 20 个 Agentic 任务。

```
Question + SQLite → 模型生成预测 SQL
Gold SQL          → 仅用于最终结果比较
```

Baseline：完整 Schema，一次生成，不修正。

Agentic：动态探索 Schema，允许执行反馈和有限修正。

5.2 实验结果

项目性能指标	Agentic	Baseline
SQL 执行成功率	✅ 20 / 20（100%）	✅ 20 / 20（100%）
与 Gold SQL 结果一致	13 / 20（65%）	15 / 20（75%）
Agent 正常完成率	19 / 20（95%）	20 / 20（100%）
平均 Agent 步数	4.4	1.0
平均 SQL 尝试	1.35	1.0
平均响应耗时	4.23 s / 题	0.91 s / 题
平均 Token	约 5,644 / 题	约 1,168 / 题

5.3 结果分析

- **可执行性稳定：** Agentic 在 20 种 Schema 上均生成可执行 SQL。
- **结果一致率 65%：** 证明跨库能力已形成，但语义精度仍需提升。
- **Baseline 当前更高：** 完整 Schema 在该样本上达到 75%。
- **Agentic 成本更高：** 动态探索带来更多步骤、耗时和 token。
- **主要错误：** 额外返回未要求字段、结果行数不准确、复杂条件理解偏差。

> 20 题是跨数据库阶段性实验，不代表完整 1,034 题 dev 准确率；本地结果一致率也不等于官方 Test Suite Accuracy。

6. 结论与改进方向

6.1 结论

本项目从 Spider 1.0 的跨数据库 Text-to-SQL 任务出发，完成了一个可运行的 C++ 查询 Agent：

- 动态适配不同 SQLite Schema；
- 形成“探索 → 生成 → 执行 → 反馈 → 修正”闭环；
- 通过 SQLite 执行层保证只读安全；
- 支持终端使用、离线演示和数据集评价；
- 20 库实验中实现 100% SQL 可执行率、65% 本地结果一致率。

6.2 改进方向

方向	计划能力
🧠 多轮上下文	`/context on/off`、`/clear`、支持“继续筛选”等追问
🔄 上下文切换	`/open` 切库自动清理，限制历史长度
📈 结果分析	统计摘要、分布、排名、异常值、分组比较
📤 分析输出	文本 / JSON，预留图表和 CSV 导出
🎯 语义精度	严格控制输出列、Top-N 和复杂条件理解

PPT 建议：7 页

页码	标题	核心内容
1	课题与摘要	标题、项目定位、核心结果
2	背景与 Spider	数据集结构、规模、任务定义
3	项目目标	数据集需求如何转化为系统能力
4	设计与实现	架构、三工具、安全层
5	运行效果	终端输入、SQL、结果与修正
6	实验结果	20 库 Agentic / Baseline 对照
7	结论与改进	主要结论、上下文、结果分析

参考文献： Yu et al. *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. EMNLP 2018.